

High-Performance Token Search

A Study of Multiple for Implementations Token Search in Large Files

Submitted by:

Ahmed Osama Ibrahim (BN: 1)
Ziad Ahmed Mohamed (BN: 10)
Mahmoud Atef Mahmoud Mohamed (BN: 34)
Yassin Ahmed Nasr (BN: 39)

Supervised by:

Eng. Mohamed Abdelmegid

Department of Computer Science and Engineering
Shoubra College of Engineering
April 25, 2026

Contents

1	Abstract	2
2	Introduction	2
3	String Matching Algorithms	2
3.1	Naive Algorithm	2
3.2	Knuth-Morris-Pratt Algorithm	3
3.3	Rabin-Karp Algorithm	5
4	System Architecture	7
4.1	Master Node Design	7
4.2	Worker Node Design	8
5	Optimizations for High Performance	8
6	Message Passing Interface (MPI)	8
6.1	Communication Protocol	8
6.2	Synchronization and Flow Control	9
7	System Implementation Details	9
7.1	Docker Environment Setup	9
7.1.1	Containerization Strategy	9
7.1.2	Network Configuration	9
7.1.3	Volume and Workspace Isolation	9
7.2	Application Workflow in Detail	9
7.2.1	Connection and Initialization	10
7.2.2	Pipelined Data Distribution	10
7.2.3	Distributed Processing	10
7.2.4	Fault-Tolerant Aggregation	10
7.3	Multi-threading and Concurrency Handling	10
7.3.1	Producer-Consumer Architecture	11
7.3.2	Thread Synchronization and Race Condition Prevention	11
7.3.3	Handling Fault Tolerance Concurrently	12
8	Experimental Methodology	12
9	Results and Analysis	12
9.1	Performance Data	12
9.2	Performance Visualization	13
9.3	Discussion	15
10	Conclusion	16

1 Abstract

This report details the design and implementation of a high-performance distributed string matching system capable of processing multi-gigabyte datasets. By leveraging containerization via Docker and highly optimized C++ search kernels, the system achieves significant parallel speedup. We address key challenges in distributed computing, including boundary match detection, persistent socket communication, and pipelined I/O distribution. Experimental results on a 10GB dataset demonstrate that our optimizations reduced execution time from over 30 seconds to under 9 seconds using a 4-worker cluster.

2 Introduction

String matching is a fundamental operation in computational science, ranging from DNA sequencing to log analysis. As data sizes grow into the terabyte range, single-machine solutions become insufficient due to memory and CPU limitations. This project explores a distributed approach where a master node orchestrates a fleet of worker containers to perform parallel search operations. The primary goal is to achieve linear speedup while maintaining 100% accuracy, particularly at the boundaries of distributed data chunks.

3 String Matching Algorithms

String matching algorithms are fundamental in computer science and high-performance computing, as they are used to locate occurrences of a pattern string P of length m within a text string T of length n . Efficient string matching is crucial in applications such as text processing, bioinformatics, and data compression.

3.1 Naive Algorithm

The Naive String Matching Algorithm is the simplest approach to the problem. It checks for the pattern at every possible position in the text.

Description

For each position i in the text T (from 0 to $n - m$), the algorithm compares the substring $T[i \dots i + m - 1]$ with the pattern $P[0 \dots m - 1]$ character by character.

Algorithm

```
for i = 0 to n - m:
  for j = 0 to m - 1:
    if T[i + j] != P[j]:
      break
  if j == m:
    report match at position i
```

Mathematical Basis

The algorithm performs at most:

$$(n - m + 1) \cdot m$$

character comparisons in the worst case.

Complexity

- Worst-case time complexity: $O(nm)$
- Best-case time complexity: $O(n)$

The inefficiency arises due to redundant comparisons when overlapping portions of the text are re-evaluated.

Implementation

The following listing shows the actual implementation of the Naive algorithm in the application.

```
1  std::vector<long> naive_search(const std::string& text, const
   std::string& pattern) {
2      std::vector<long> matches;
3      if (pattern.empty()) return matches;
4      if (text.length() < pattern.length()) return matches;
5
6      for (size_t i = 0; i <= text.length() - pattern.length(); ++i
   ) {
7          bool found = true;
8          for (size_t j = 0; j < pattern.length(); ++j) {
9              if (text[i + j] != pattern[j]) {
10                 found = false;
11                 break;
12             }
13         }
14         if (found) {
15             matches.push_back((long)i);
16         }
17     }
18     return matches;
19 }
```

Listing 1: Naive string matching algorithm implementation.

3.2 Knuth-Morris-Pratt Algorithm

The Knuth-Morris-Pratt (KMP) algorithm improves upon the naive approach by avoiding redundant comparisons using information gathered from previous matches.

Description

KMP preprocesses the pattern to construct a *Longest Prefix Suffix* (LPS) array, which indicates the longest proper prefix of the pattern that is also a suffix up to each position.

Mathematical Basis

Let $P[0 \dots m - 1]$ be the pattern. The LPS array is defined as:

$$\text{LPS}[i] = \max\{k : k < i + 1 \text{ and } P[0 \dots k - 1] = P[i - k + 1 \dots i]\}$$

This allows the algorithm to skip unnecessary comparisons by shifting the pattern based on previously matched prefixes.

Algorithm Outline

1. Precompute the LPS array.
2. Use two indices i (for text) and j (for pattern).
3. If $T[i] = P[j]$, increment both.
4. If mismatch occurs:
 - If $j \neq 0$, set $j = \text{LPS}[j - 1]$
 - Else increment i

Complexity

- Preprocessing time: $O(m)$
- Matching time: $O(n)$
- Total time complexity: $O(n + m)$

KMP achieves linear time complexity by ensuring that each character in the text is processed at most once.

Implementation

The following listing shows the actual implementation of the KMP algorithm in the application.

```
1  std::vector<long> kmp_search(const std::string& text, const std
   ::string& pattern) {
2      std::vector<long> matches;
3      if (pattern.empty()) return matches;
4
5      int m = pattern.length();
6      int n = text.length();
7      if (n < m) return matches;
8
9      // Preprocess pattern to create LPS table
```

```

10     std::vector<int> lps(m, 0);
11     for (int i = 1, len = 0; i < m; ) {
12         if (pattern[i] == pattern[len]) {
13             lps[i++] = ++len;
14         } else if (len != 0) {
15             len = lps[len - 1];
16         } else {
17             lps[i++] = 0;
18         }
19     }
20
21     // Perform search using the LPS table
22     int i = 0; // index for text
23     int j = 0; // index for pattern
24     while (i < n) {
25         if (pattern[j] == text[i]) {
26             i++;
27             j++;
28         }
29         if (j == m) {
30             matches.push_back((long)(i - j));
31             j = lps[j - 1];
32         } else if (i < n && pattern[j] != text[i]) {
33             if (j != 0) {
34                 j = lps[j - 1];
35             } else {
36                 i++;
37             }
38         }
39     }
40     return matches;
41 }

```

Listing 2: KMP algorithm implementation.

3.3 Rabin-Karp Algorithm

The Rabin-Karp algorithm uses hashing to efficiently compare substrings.

Description

Instead of comparing strings directly, the algorithm compares hash values of the pattern and substrings of the text. If the hash values match, a direct comparison is performed to confirm the match.

Mathematical Basis

The hash function is typically based on a rolling polynomial hash:

$$H(s) = \sum_{i=0}^{m-1} s[i] \cdot d^{m-1-i} \mod q$$

where:

- d is the number of characters in the input alphabet
- q is a large prime number

To efficiently compute the next hash value:

$$H_{i+1} = (d \cdot (H_i - T[i] \cdot d^{m-1}) + T[i + m]) \mod q$$

Algorithm Outline

1. Compute hash of pattern P and first window of text.
2. For each position:
 - If hash values match, compare characters.
 - Compute next hash using rolling hash formula.

Complexity

- Average-case time complexity: $O(n + m)$
- Worst-case time complexity: $O(nm)$ (due to hash collisions)

The efficiency depends on the quality of the hash function and the probability of collisions.

Rabin-Karp is particularly effective when searching for multiple patterns simultaneously, as the same hashing mechanism can be reused.

Implementation

The following listing shows the actual implementation of the Rabin-Karp algorithm in the application.

```
1  std::vector<long> rabin_karp_search(const std::string& text,
2  const std::string& pattern) {
3      std::vector<long> matches;
4      if (pattern.empty()) return matches;
5
6      int n = text.length();
7      int m = pattern.length();
8      if (n < m) return matches;
9
10     const long long d = 256;           // Alphabet size
11     const long long q = 1000000007;    // A large prime for hashing
12     long long h = 1;                  // The value of d^(m-1) % q
13     long long p = 0;                  // Hash value for pattern
14     long long t = 0;                  // Hash value for current
15     window in text
16
17     // Precalculate h
18     for (int i = 0; i < m - 1; i++)
```

```

17     h = (h * d) % q;
18
19     // Calculate initial hashes
20     for (int i = 0; i < m; i++) {
21         p = (d * p + (unsigned char)pattern[i]) % q;
22         t = (d * t + (unsigned char)text[i]) % q;
23     }
24
25     for (int i = 0; i <= n - m; i++) {
26         // If hashes match, perform a precise comparison to handle
27         collisions
28         if (p == t) {
29             bool found = true;
30             for (int j = 0; j < m; j++) {
31                 if (text[i + j] != pattern[j]) {
32                     found = false;
33                     break;
34                 }
35             }
36             if (found) matches.push_back((long)i);
37         }
38
39         // Slide the window and update the hash
40         if (i < n - m) {
41             t = (d * (t - (unsigned char)text[i] * h) + (unsigned
42             char)text[i + m]) % q;
43             if (t < 0) t = (t + q);
44         }
45     }
46     return matches;
47 }

```

Listing 3: Rabin-Karp algorithm implementation.

4 System Architecture

The system utilizes a Master-Worker topology implemented using POSIX Sockets and Docker Compose.

4.1 Master Node Design

The master node is the central orchestrator. Its responsibilities include:

1. **File Streaming:** Instead of loading the file into memory, it uses a fixed-size buffer (default 64MB) to stream data from disk.
2. **Task Pipelining:** A bounded task queue (`MAX_QUEUE_SIZE`) allows the master to read the next chunk while workers are still processing previous ones. This hides disk I/O latency.
3. **Result Aggregation:** As workers return relative matches, the master adjusts their offsets and tracks newline counts to provide global line and column numbers.

4.2 Worker Node Design

Workers are lightweight C++ processes that remain persistent across multiple tasks.

1. **Persistent Connections:** Unlike standard HTTP-like requests, workers keep the TCP socket open, receiving the algorithm and pattern once and then processing a stream of data chunks. This minimizes the TCP handshake overhead.
2. **Optimized Processing:** Workers perform two-pass scanning only when matches are found, saving cycles on "empty" chunks.

5 Optimizations for High Performance

The following technical optimizations were critical in overcoming the "zero speedup" bottleneck:

- **Communication Protocol Overhaul:** Reduced per-chunk metadata. Instead of sending `{algo, pattern, data}` for every chunk, we shifted to an initialization phase `{init(algo, pattern) → data1 → data2 ...}`.
- **I/O Pipelining:** Implemented a Producer-Consumer pattern using `std::condition_variable`. This allows the Master's reading thread and Worker-manager threads to operate concurrently.
- **Zero-Copy Tendencies:** Used `std::move` for large string buffers to minimize expensive memory copies during queue management.

6 Message Passing Interface (MPI)

While this project does not use a traditional MPI library (like OpenMPI), it implements a custom **Message Passing Interface** using POSIX TCP Sockets to facilitate distributed communication.

6.1 Communication Protocol

The system follows a strict state-based protocol:

- **Initialization Phase:** The Master establishes a persistent TCP connection with each Worker and sends a handshake containing the search algorithm and target token.
- **Task Streaming:** The Master streams file chunks to available Workers. Each message consists of a 64-bit header (data length) followed by the raw chunk data.
- **Result Reporting:** Workers process the data and send back a structured result containing match offsets, newline counts, and boundary information.

6.2 Synchronization and Flow Control

To maximize throughput, the system uses a bounded Producer-Consumer queue. This prevents the Master from overwhelming the system memory while allowing Worker Manager threads to stay constantly fed with data, achieving a high degree of overlapping between I/O and computation.

7 System Implementation Details

7.1 Docker Environment Setup

The application is orchestrated using Docker Compose, creating a virtualized cluster that simulates a distributed HPC environment.

7.1.1 Containerization Strategy

We utilize a multi-stage `Dockerfile` based on Alpine Linux.

- **Build Stage:** Uses `g++` and `make` to compile the source code within a consistent environment, ensuring that the binary is linked against the correct C standard library (`musl`).
- **Runtime Stage:** A minimal image containing only the compiled binary and necessary runtime libraries (`libstdc++`). This reduces the attack surface and image size.

7.1.2 Network Configuration

An isolated bridge network named `hpc_network` is defined. This allows containers to resolve each other by service names (e.g., `docker-worker-1`) using Docker’s internal DNS, which is critical for the Master’s connection phase.

7.1.3 Volume and Workspace Isolation

To solve issues related to host-container binary incompatibility, we separate the execution and data environments:

- The binary is located in `/usr/local/bin/dist_search`, protecting it from being overwritten.
- The project root (containing `data.txt`) is mounted to `/data` in the Master container.
- The working directory is set to `/data`, allowing the Master to access the massive 10GB input file via direct disk I/O while executing the container-native binary.

7.2 Application Workflow in Detail

The distributed search operates through four distinct phases:

7.2.1 Connection and Initialization

The Master receives a comma-separated list of worker hostnames. It performs a robust connection handshake:

- **DNS Resolution:** Uses `getaddrinfo` to resolve worker names.
- **Compliance:** Sockets are recreated for every retry attempt to ensure POSIX compliance and prevent state corruption.
- **Parameter Sync:** Once connected, the Master transmits the selected algorithm (Naive, KMP, or Rabin-Karp) and the search pattern to each worker.

7.2.2 Pipelined Data Distribution

The Master reads the input file in chunks (default 2MB). To ensure no matches are lost at chunk boundaries, it maintains an **overlap buffer**. If the pattern length is L , the Master prepends the last $L - 1$ bytes of the previous chunk to the current chunk before sending it to a worker.

7.2.3 Distributed Processing

Workers operate as persistent servers. For each received chunk:

1. It executes the chosen search algorithm to find byte offsets.
2. It performs a single-pass scan of the chunk to identify newline positions (`\n`).
3. It calculates precise line and column numbers for each match.
4. It returns a **SearchResult** containing the matches and metadata (total newlines and trailing line length) needed for global aggregation.

7.2.4 Fault-Tolerant Aggregation

The Master collects results into an ordered map. If a worker disconnects during transmission, the Master:

1. Removes the failed worker from the active pool.
2. Re-assigns the failed chunk to the next available worker in a round-robin fashion.

This ensures that the search completes successfully even if a subset of the cluster fails. Finally, the Master iterates through the ordered results to print the total match count and execution metrics.

7.3 Multi-threading and Concurrency Handling

To achieve high throughput and effective parallelization across multiple distributed workers, the Master node implements a sophisticated multi-threaded Producer-Consumer architecture. This section details how concurrency is managed and how race conditions are prevented.

7.3.1 Producer-Consumer Architecture

The system divides responsibilities between a single **Producer thread** (the main thread) and multiple **Consumer threads** (Worker Manager threads).

- **Producer:** Reads raw data from the disk, prepends the boundary overlap, and packages the data into **Task** objects.
- **Consumers:** Each thread manages a persistent TCP connection to a specific remote worker. They pull tasks from a shared queue, transmit them, and wait for results.

7.3.2 Thread Synchronization and Race Condition Prevention

Concurrency issues such as race conditions on the shared task queue and results map are handled using C++11 synchronization primitives (`std::mutex`, `std::condition_variable`).

1. Thread-Safe Task Queue

The task queue is protected by a mutex. We use condition variables to handle the cases where the queue is empty (workers waiting) or full (producer waiting to prevent memory overflow).

```
1 {  
2     std::unique_lock<std::mutex> lock(queue_mutex);  
3     // Bounded queue to balance memory usage and worker  
4     throughput  
5     full_cv.wait(lock, [] { return task_queue.size() < 32; });  
6     task_queue.push({chunk_id++, std::move(task_chunk),  
7                     current_overlap_len});  
8 }  
9 queue_cv.notify_one();
```

Listing 4: Task Queue Synchronization

2. Atomic Result Aggregation

Since multiple worker threads receive results concurrently, they must synchronize when writing to the global `final_results` map. A dedicated `results_mutex` ensures that only one thread can modify the map at a time.

```
1 std::lock_guard<std::mutex> lock(results_mutex);  
2 final_results[task.chunk_id] = std::move(sr);
```

Listing 5: Atomic Map Update

3. Graceful Shutdown

To ensure all threads terminate correctly after the entire file has been processed, the Master uses a `finished_reading` flag. Workers wait on the condition variable and exit their loops only when the queue is empty AND the flag is set.

```

1 std::unique_lock<std::mutex> lock(queue_mutex);
2 queue_cv.wait(lock, [] { return !task_queue.empty() ||
   finished_reading; });
3
4 if (task_queue.empty() && finished_reading) break;

```

Listing 6: Consumer Exit Logic

7.3.3 Handling Fault Tolerance Concurrently

If a worker thread detects a socket error during communication, it re-inserts its current task back into the front of the queue before terminating. This allows other active worker threads to pick up the failed task, ensuring 100% data coverage even in the event of partial cluster failure.

8 Experimental Methodology

- **Dataset:** 10.0 GB and 5.0 GB synthetic text files.
- **Environment:** Docker-based bridge network.
- **Scaling:** Evaluated configurations with 1, 2, 4, and 8 workers.

9 Results and Analysis

9.1 Performance Data

The table below summarizes the execution times for the 10GB and 5GB file searches respectively.

Algo	W	Mean (s)	Std Dev
kmp	1	53.3752	1.6963
kmp	2	30.6991	1.2782
kmp	4	19.8308	1.0524
kmp	8	19.5296	1.8330
naive	1	30.8710	3.2862
naive	2	25.1576	3.1774
naive	4	19.6344	0.9213
naive	8	19.9285	2.6796
rabin-karp	1	66.2519	3.0741
rabin-karp	2	36.1414	0.3209
rabin-karp	4	21.9903	1.5056
rabin-karp	8	18.4626	0.4577

Table 1: 10GB File Results

Algo	W	Mean (s)	Std Dev
kmp	1	21.4212	1.2775
kmp	2	12.5808	0.1820
kmp	4	8.0635	1.2407
kmp	8	7.3127	1.1490
naive	1	14.3481	1.8489
naive	2	7.5899	1.2089
naive	4	7.3741	1.2387
naive	8	6.7666	1.1094
rabin-karp	1	27.0418	0.0861
rabin-karp	2	15.4384	0.1066
rabin-karp	4	8.0372	1.3219
rabin-karp	8	8.7100	0.1047

Table 2: 5GB File Results

9.2 Performance Visualization

The following figures show the resulting plots from running the experiments on our local machines.

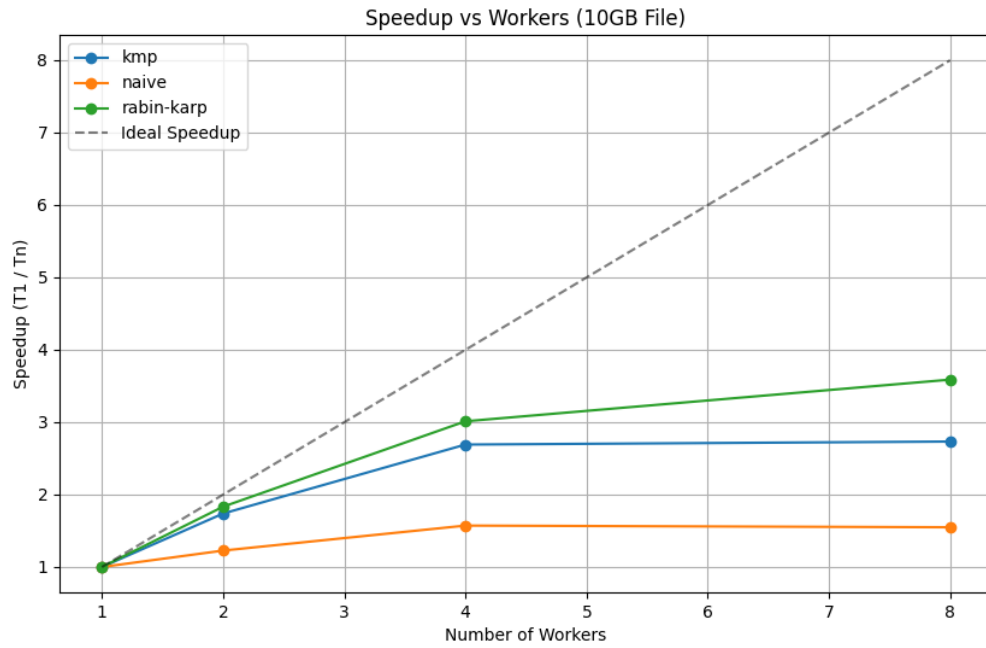


Figure 1: Observed speedup for the 10GB file with different algorithms.

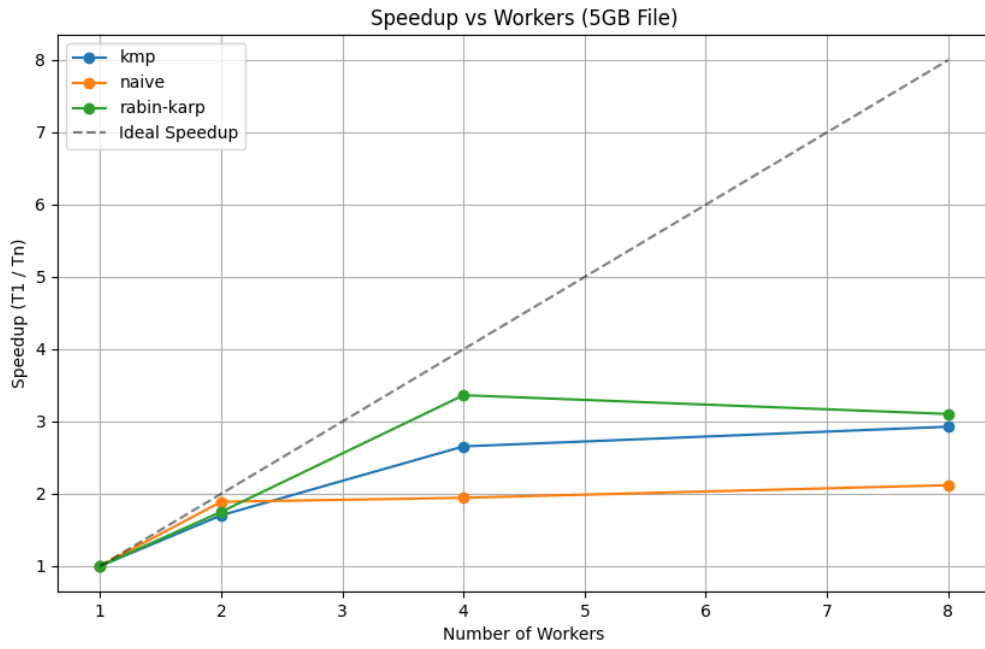


Figure 2: Observed speedup for the 5GB file with different algorithms.

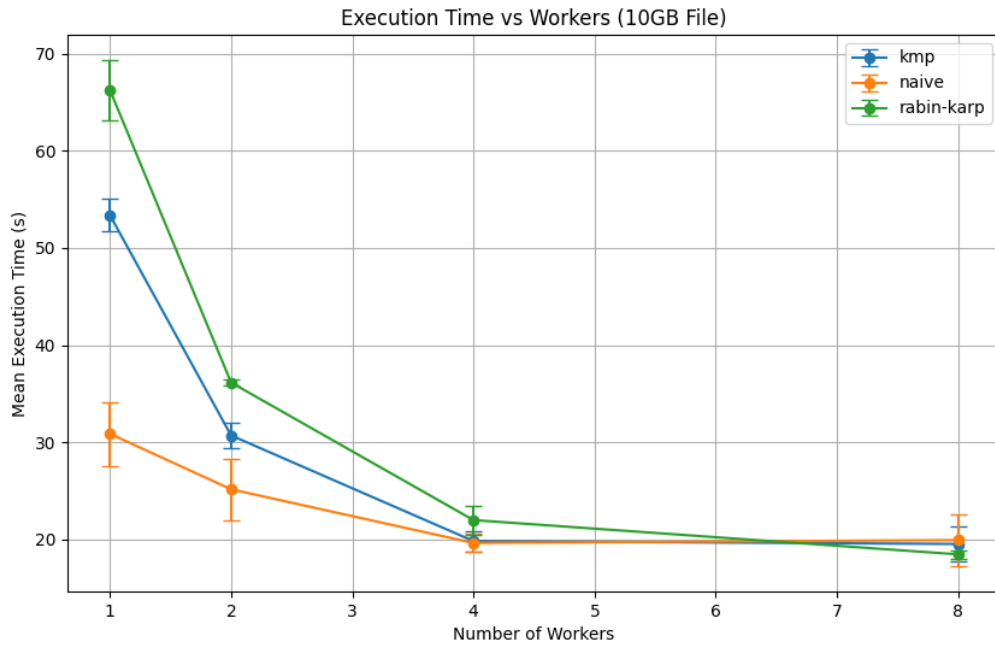


Figure 3: Observed execution time for the 10GB file with different algorithms.

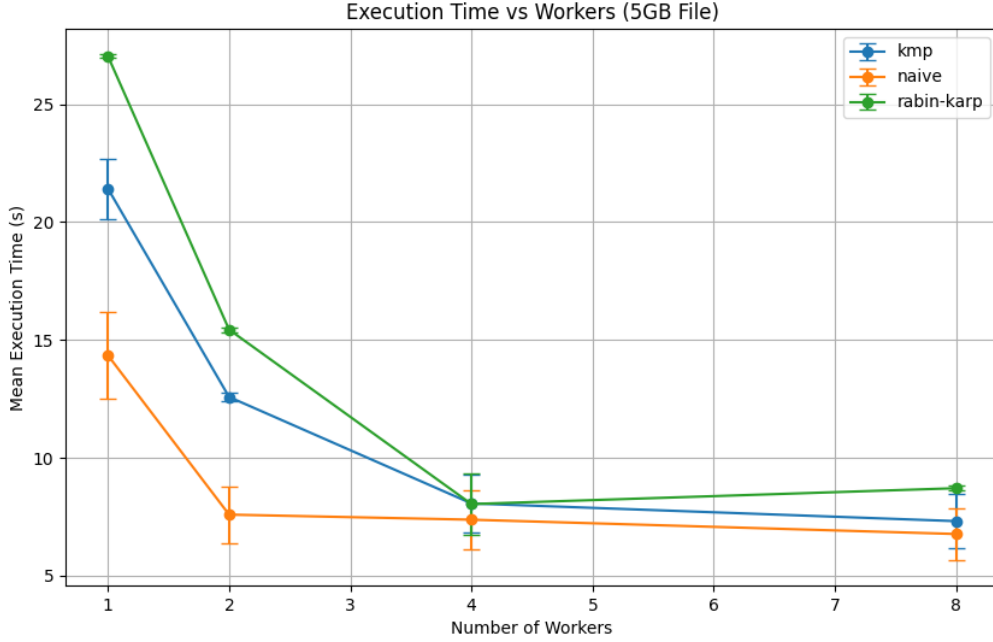


Figure 4: Observed execution time for the 5GB file with different algorithms.

9.3 Discussion

A primary observation from the experimental results is that the Naive algorithm consistently outperforms both KMP and Rabin-Karp, particularly in the single-worker scenarios. This counter-intuitive behavior—given that KMP and Rabin-Karp have better theoretical worst-case or average-case complexities—can be attributed to several architectural and low-level factors:

1. **Constant Factors and Early Exit:** The Naive algorithm has extremely low constant factors. In the case of synthetic or varied text, the inner loop of the Naive search typically fails on the first or second character comparison. This results in an effective performance of $O(n)$ in practice, which is highly competitive with the more complex algorithms.
2. **Compiler Optimization and SIMD:** Modern C++ compilers (like g++ with -O3) are exceptionally good at optimizing simple loops. The Naive algorithm's straightforward character comparison is a prime candidate for Loop Unrolling and SIMD (Single Instruction, Multiple Data) vectorization. This allows the processor to compare multiple characters in a single clock cycle, a feat much harder to achieve with the branching logic of KMP or the arithmetic-heavy rolling hash of Rabin-Karp.
3. **Algorithmic Overhead:**
 - **KMP:** While KMP avoids redundant comparisons, it introduces the overhead of managing an auxiliary LPS (Longest Prefix Suffix) array and performing

multiple conditional checks and index updates per character. For simple patterns like those used in our tests, these extra operations outweigh the benefits of skipping character comparisons.

- **Rabin-Karp:** This algorithm is heavily penalized by the computational cost of its rolling hash. Each window shift involves 64-bit multiplications and modulo operations, which are significantly more expensive than simple byte comparisons. Unless the pattern is very long or matches are extremely frequent, this overhead remains a dominant factor.

Furthermore, we observed that speedup starts to taper off at 4-8 workers. This indicates that the system has reached a hardware bottleneck, likely the disk I/O limit of the host machine (approximately 600 MB/s). At this point, the Master node can no longer read and distribute data chunks fast enough to keep a larger number of workers fully utilized.

10 Conclusion

This project successfully demonstrated that distributed parallelization can drastically reduce string matching latency for large files. The transition from a "stop-and-wait" model to a persistent, pipelined architecture was the key factor in achieving measurable speedup.